

```

# =====
# 1. Depth First Search (DFS)
# =====
import collections

def dfs(g, n, seen, d):
    if n not in seen:
        seen.append(n)
        for i in g[n]:
            if seen[-1] is d:
                break
            dfs(g, i, seen, d)
    return seen

graph = {
    'M': ['R', 'Q', 'N'],
    'N': ['M', 'Q', 'O'],
    'O': ['N', 'P']
}
print(dfs(graph, 'M', [], 'P'))

```

```

# =====
# 2. Breadth First Search (BFS) & Shortest Path
# =====
import collections

def bfs(graph, root):
    seen, queue = set([root]), collections.deque([root])
    while queue:
        vertex = queue.popleft()
        visit(vertex)
        for node in graph[vertex]:
            if node not in seen:
                seen.add(node)
                queue.append(node)

def all_path(st, end, gr):
    todo = [[st, [st]]]
    while len(todo):
        node, path = todo.pop(0)
        for next_node in gr[node]:
            if next_node in path:
                continue
            elif next_node == end:

```

```

        yield path + [next_node]
    else:
        todo.append([next_node, path + [next_node]])

def visit(n):
    print(n)

def bfs_shortest_path(graph, source, destination):
    checked = []
    queue = [[source]]
    if source == destination:
        return "Source is Destination"
    while queue:
        path = queue.pop(0)
        node = path[-1]
        if node not in checked:
            neighbours = graph[node]
            for neighbour in neighbours:
                new_path = list(path)
                new_path.append(neighbour)
                queue.append(new_path)
                if neighbour == destination:
                    return new_path
            checked.append(node)
    return "Path Does Not Exist"

Graph = {'A': ['B', 'D'], 'B': ['A', 'C'], 'C': ['B', 'G'], 'D': ['A', 'E', 'H'], 'E': ['D', 'F'], 'F': ['E', 'G'], 'G': ['C', 'F'], 'H': ['D']}

# =====
# 3. N-Queen Problem
# =====

def print_board(board):
    for row in board:
        print(" ".join(row))
    print()

def check_e(board, row, col):
    for i in range(row):
        if board[i][col] == 'Q':
            return False
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 'Q':

```

```

        return False
    i -= 1
    j -= 1
    i, j = row, col
    while i >= 0 and j < len(board):
        if board[i][j] == 'Q':
            return False
        i -= 1
        j += 1
    return True

```

```

def solve_queens(board, row, n):
    if row == n:
        print_board(board)
        return True
    for col in range(n):
        if check_e(board, row, col):
            board[row][col] = 'Q'
            if solve_queens(board, row + 1, n):
                return True
            board[row][col] = '.'
    return False

```

```

def queens():
    n = int(input("Enter value of N: "))
    board = []
    for i in range(n):
        row = []
        for j in range(n):
            row.append(".")
        board.append(row)
    if not solve_queens(board, 0, n):
        print("No Solution found")

```

queens()

```

# =====
# 4. Tower of Hanoi
# =====

```

```

def tower(n, source, auxillary, destination):
    if n == 1:
        print("Move disk 1 from source", source, "to destination", destination)
        return
    tower(n-1, source, destination, auxillary)

```

```
print("Move disk", n, "from", source, "to destination", destination)
tower(n-1, auxillary, source, destination)
```

```
n = int(input("Enter number of disks: "))
tower(n, 'A', 'B', 'C')
```

```
# =====
```

```
# 5. Alpha-Beta Search
```

```
# =====
```

```
tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [4, 3],
    'E': [6, 2],
    'F': [2, 1],
    'G': [9, 5]
}
```

```
def minimax_alpha_beta(node, depth, alpha, beta, max_player):
```

```
    if depth == 0 or node not in tree:
```

```
        if node in tree:
```

```
            return tree[node][0]
```

```
        else:
```

```
            return node
```

```
    if max_player:
```

```
        value = -float('inf')
```

```
        for child in tree[node]:
```

```
            value = max(value, minimax_alpha_beta(child, depth-1, alpha, beta, False))
```

```
            alpha = max(alpha, value)
```

```
            if alpha >= beta:
```

```
                print(f"Pruning branch at node {node}")
```

```
                break
```

```
        return value
```

```
    else:
```

```
        value = float('inf')
```

```
        for child in tree[node]:
```

```
            value = min(value, minimax_alpha_beta(child, depth-1, alpha, beta, True))
```

```
            beta = min(beta, value)
```

```
            if beta <= alpha:
```

```
                print(f"Pruning branch at node {node}")
```

```
                break
```

```
        return value
```

```
best_score = minimax_alpha_beta('A', 2, -float('inf'), float('inf'), True)
print(f"The best score is {best_score}")
```

```
# =====
```

```
# 6. Hill Climbing
```

```
# =====
```

```
import random
```

```
distance = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
```

```
def get_cost(tour):
    cost = 0
    for i in range(len(tour)):
        cost += distance[tour[i-1]][tour[i]]
    print("Cost of tour:", cost)
    return cost
```

```
def get_neighbor(tour):
    a, b = random.sample(range(len(tour)), 2)
    tour[a], tour[b] = tour[b], tour[a]
    return tour
```

```
def hill_climb():
    current = [0, 1, 2, 3]
    random.shuffle(current)
    current_cost = get_cost(current)
    print("Starting tour:", current, "cost:", current_cost)
```

```
    for i in range(100):
        neighbor = current[:]
        print("Current neighbor:", neighbor)
        neighbor = get_neighbor(neighbor)
        print("Connected neighbor:", neighbor)
        neighbor_cost = get_cost(neighbor)
```

```
        if neighbor_cost < current_cost:
            current = neighbor[:]
            print("Current neighbor:", current)
```

```

        current_cost = neighbor_cost
        print("Current cost:", current_cost)
        print("Better tour found:", current, "cost:", current_cost)

    return current, current_cost

best_tour, best_cost = hill_climb()
print("Best tour:", best_tour)
print("Best cost:", best_cost)

# =====
# 7. Greedy Best First Search
# =====
graph_greedy = {
    "A": {"D": 1, "B": 2},
    "B": {"C": 2},
    "C": {"G": 3},
    "D": {"B": 3, "G": 2, "E": 4},
    "E": {"G": 2},
    "S": {"D": 1, "A": 2, "E": 3},
    "G": {}
}

def greedy_search_rec(graph, prev, dst, path, h):
    print("Connected nodes of current node", prev, "with h(n) values")
    q = {}
    for n in graph[prev]:
        if n not in path:
            q[n] = graph[prev][n]
            print(n, ":", q[n])

    while q:
        mn = min(q, key=q.get)
        q.pop(mn)
        print("Taking minimum h(n) vertex:", mn)
        if dst == mn:
            return path + [dst]
        new_path = greedy_search_rec(graph, mn, dst, path + [mn], h)
        if new_path:
            return new_path
    return None

source = input("Enter source vertex: ")
dest = input("Enter destination vertex: ")

```

```

path = greedy_search_rec(graph_greedy, source, dest, [source], {})
if path:
    print(path)
else:
    print("Path not found!")

```

```

# =====
# 8. A* Algorithm
# =====

```

```

graph_astar = {
    "a": {"b": 1, "d": 2, "e": 3},
    "b": {"c": 2},
    "c": {"f": 3},
    "d": {"f": 2},
    "e": {"f": 3},
    "f": {}
}

```

```

heuristics = {'a': 0, 'b': 1, 'c': 2, 'd': 0, 'e': 0, 'f': 0}

```

```

def get_lowest_f_node(open_list, f_scores):
    lowest_node = open_list[0]
    lowest_f = f_scores[lowest_node]
    for node in open_list:
        if f_scores[node] < lowest_f:
            lowest_f = f_scores[node]
            lowest_node = node
    return lowest_node

```

```

def a_star_algorithm_with_trace(graph, heuristics, start, goal):
    print(f"Enter source vertex: {start.upper()}")
    print(f"Enter destination vertex: {goal.upper()}")
    open_list = [start]
    closed_list = set()

```

```

    g_scores = {node: float('inf') for node in graph}
    g_scores[start] = 0
    f_scores = {node: float('inf') for node in graph}
    f_scores[start] = heuristics[start]
    came_from = {}

```

```

    while len(open_list) > 0:
        current_node = get_lowest_f_node(open_list, f_scores)

```

```

if current_node != start:
    print(f"Taking minimum f(n) vertex: {current_node.upper()}")

if current_node == goal:
    path = []
    curr = current_node
    while curr in came_from:
        path.append(curr)
        curr = came_from[curr]
    path.append(start)
    path.reverse()
    formatted_path = " -> ".join([f"{n.upper()}" for n in path])
    print(formatted_path)
    return path

open_list.remove(current_node)
closed_list.add(current_node)
updated_neighbors = {}

for neighbor, weight in graph[current_node].items():
    if neighbor in closed_list:
        continue

    tentative_g_score = g_scores[current_node] + weight
    if neighbor not in open_list:
        open_list.append(neighbor)
    elif tentative_g_score >= g_scores[neighbor]:
        continue

    came_from[neighbor] = current_node
    g_scores[neighbor] = tentative_g_score
    f_scores[neighbor] = g_scores[neighbor] + heuristics[neighbor]
    updated_neighbors[neighbor] = f_scores[neighbor]

if updated_neighbors:
    print(f"Connected nodes of current node {current_node.upper()} with f(n) values")
    for n, val in updated_neighbors.items():
        print(f"{n.upper()} : {val}")
return None

a_star_algorithm_with_trace(graph_astar, heuristics, 'a', 'f')

# =====
# 9. Water Jug Problem

```

```

# =====
from collections import deque

def is_visited(state, visited):
    return state in visited

def water_jug_bfs():
    max_a, max_b = 6, 5
    visited = set()
    queue = deque()
    queue.append((0, 0))

    while queue:
        a, b = queue.popleft()
        if (a, b) in visited:
            continue

        visited.add((a, b))
        print(f"Jug A: {a}L, Jug B: {b}L")

        if a == 4 or b == 4:
            print("Found a solution!")
            return

        possible_states = [
            (max_a, b),
            (a, max_b),
            (0, b),
            (a, 0),
            (a - min(a, max_b - b), b + min(a, max_b - b)),
            (a + min(max_a - a, b), b - min(max_a - a, b))
        ]

        for state in possible_states:
            if state not in visited:
                queue.append(state)

    print("No solution found.")

water_jug_bfs()

# =====
# 10. Travelling Salesman Problem
# =====

```

```
from itertools import permutations
```

```
dist = [  
    [0, 10, 15, 20],  
    [10, 0, 35, 25],  
    [15, 35, 0, 30],  
    [20, 25, 30, 0]  
]
```

```
n_cities = len(dist)  
cities = range(1, n_cities)  
min_distance = float('inf')  
best_path = None
```

```
for path in permutations(cities):  
    current_path = (0,) + path + (0,)  
    distance_tsp = 0  
    for i in range(len(current_path) - 1):  
        distance_tsp += dist[current_path[i]][current_path[i+1]]  
  
    if distance_tsp < min_distance:  
        min_distance = distance_tsp  
        best_path = current_path
```

```
print("Shortest distance is:", min_distance)  
print("Best path:", best_path)
```

```
# =====  
# 11. Missionaries and Cannibals  
# =====  
from collections import deque
```

```
moves_mc = [(2, 0), (0, 2), (1, 1), (1, 0), (0, 1)]
```

```
def is_valid_mc(m_left, c_left, m_right, c_right):  
    if m_left < 0 or c_left < 0 or m_right < 0 or c_right < 0:  
        return False  
    if (m_left > 0 and m_left < c_left) or (m_right > 0 and m_right < c_right):  
        return False  
    return True
```

```
def solve_mc():  
    start = (3, 3, 1)  
    goal = (0, 0, 0)
```

```

queue = deque([(start, [start])])
visited = set()

while queue:
    (m_left, c_left, boat), path = queue.popleft()

    if (m_left, c_left, boat) in visited:
        continue

    visited.add((m_left, c_left, boat))

    if (m_left, c_left, boat) == goal:
        return path

    for m, c in moves_mc:
        if boat == 1:
            new_m_left = m_left - m
            new_c_left = c_left - c
            new_boat = 0
        else:
            new_m_left = m_left + m
            new_c_left = c_left + c
            new_boat = 1

        new_m_right = 3 - new_m_left
        new_c_right = 3 - new_c_left

        if is_valid_mc(new_m_left, new_c_left, new_m_right, new_c_right):
            new_state = (new_m_left, new_c_left, new_boat)
            if new_state not in visited:
                queue.append((new_state, path + [new_state]))

    return None

steps = solve_mc()
if steps:
    for i, (m, c, b) in enumerate(steps):
        side = "left" if b == 1 else "Right"
        print(f"Step {i}: Missionaries left: {m}, Cannibals left: {c}, Boat on: {side}")
else:
    print("No Solution found")

# =====
# 12. Number Puzzle (8-Puzzle)

```

```

# =====
from collections import deque

goal_puzzle = list("123456780")
moves_puzzle = {
    0: [1, 3],
    1: [0, 2, 4],
    2: [1, 5],
    3: [0, 4, 6],
    4: [1, 3, 5, 7],
    5: [2, 4, 8],
    6: [3, 7],
    7: [4, 6, 8],
    8: [5, 7]
}

def bfs_puzzle(start):
    visited = set()
    queue = deque([(start, [start])])

    while queue:
        state, path = queue.popleft()
        if state == goal_puzzle:
            return path
        if tuple(state) in visited:
            continue

        visited.add(tuple(state))
        zero = state.index('0')

        for move in moves_puzzle[zero]:
            new_state = list(state)
            new_state[zero], new_state[move] = new_state[move], new_state[zero]
            queue.append((new_state, path + [new_state]))

    return None

start_puzzle = list("123405678")
solution = bfs_puzzle(start_puzzle)
if solution:
    print("Steps to solve:")
    for s in solution:
        print("".join(s[0:3]))
        print("".join(s[3:6]))

```

```

        print("".join(s[6:9]))
        print()
    else:
        print("No Solution found")

# =====
# 13. Shuffle Deck of Cards
# =====
import random

suits = ["Hearts", "Diamonds", "Clubs", "Spades"]
ranks = ["A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K"]
deck = [rank + " of " + suit for suit in suits for rank in ranks]

print("Deck before Shuffle:")
print(deck)
input("Press Enter to Shuffle the deck... ")
random.shuffle(deck)

print("Deck after Shuffle:")
for card in deck:
    print(card)

# =====
# 14. Tic-Tac-Toe Game
# =====
board_ttt = [' ' for _ in range(9)]
player_ttt = 'X'

def show_board():
    print(f" {board_ttt[0]} | {board_ttt[1]} | {board_ttt[2]} ")
    print(f" {board_ttt[3]} | {board_ttt[4]} | {board_ttt[5]} ")
    print(f" {board_ttt[6]} | {board_ttt[7]} | {board_ttt[8]} ")

def is_winner(p):
    return ((board_ttt[0] == p and board_ttt[1] == p and board_ttt[2] == p) or
            (board_ttt[3] == p and board_ttt[4] == p and board_ttt[5] == p) or
            (board_ttt[6] == p and board_ttt[7] == p and board_ttt[8] == p) or
            (board_ttt[0] == p and board_ttt[3] == p and board_ttt[6] == p) or
            (board_ttt[1] == p and board_ttt[4] == p and board_ttt[7] == p) or
            (board_ttt[2] == p and board_ttt[5] == p and board_ttt[8] == p) or
            (board_ttt[0] == p and board_ttt[4] == p and board_ttt[8] == p) or
            (board_ttt[2] == p and board_ttt[4] == p and board_ttt[6] == p))

```

```

def is_tie():
    return ' ' not in board_ttt

def game():
    global player_ttt
    while True:
        show_board()
        try:
            move = int(input(f"Player {player_ttt} enter position (0-8): "))
            if 0 <= move <= 8 and board_ttt[move] == ' ':
                board_ttt[move] = player_ttt
                if is_winner(player_ttt):
                    show_board()
                    print(f"Player {player_ttt} wins!")
                    break
                elif is_tie():
                    show_board()
                    print("It's a tie!")
                    break
                if player_ttt == 'X':
                    player_ttt = 'O'
                else:
                    player_ttt = 'X'
            else:
                print("Invalid move!")
        except ValueError:
            print("Invalid input. Enter a number 0-8")

```

```

game()

```

```

# =====
# 15. Constraint Satisfaction (Map Coloring)
# =====

```

```

import itertools

```

```

variables_map = ["A", "B", "C", "D"]
colors_map = ["Red", "Blue", "Yellow", "Green"]
all_assignment = itertools.product(colors_map, repeat=len(variables_map))

```

```

def valid_map(i):
    A, B, C, D = i
    return (A != B) and (B != C) and (A != C) and (C != D)

```

```

solutions_map = []

```

```

for i in all_assignment:
    if valid_map(i):
        solutions_map.append(dict(zip(variables_map, i)))

print("Variables Coloring of the map:")
for sol in solutions_map:
    print(sol)

# =====
# 16. Associative and Distributive Laws
# =====

# Associative law of multiplication
a1 = int(input("Enter value a (multiply): "))
b1 = int(input("Enter value b (multiply): "))
c1 = int(input("Enter value c (multiply): "))
lhs1 = (a1 * b1) * c1
rhs1 = a1 * (b1 * c1)
print("LHS =", lhs1)
print("RHS =", rhs1)
if lhs1 == rhs1:
    print("Associative law is true")
else:
    print("Associative law is false")

# Associative law of Logical AND
a2 = bool(int(input("enter value of a (AND): ")))
b2 = bool(int(input("enter value of b (AND): ")))
c2 = bool(int(input("enter value of c (AND): ")))
lhs2 = (a2 and b2) and c2
rhs2 = a2 and (b2 and c2)
print("LHS=", lhs2)
print("RHS=", rhs2)
if lhs2 == rhs2:
    print("Associative law is True")
else:
    print("False")

# Associative law of Logical OR
a3 = bool(int(input("enter value 0 or 1 (OR): ")))
b3 = bool(int(input("enter value 0 or 1 (OR): ")))
c3 = bool(int(input("enter value 0 or 1 (OR): ")))
lhs3 = (a3 or b3) or c3
rhs3 = a3 or (b3 or c3)

```

```
print("LHS=", lhs3)
print("RHS=", rhs3)
if lhs3 == rhs3:
    print("Associative law is True")
else:
    print("Associative law is false")
```

```
# Associative law of addition (+)
a4 = int(input("enter value of a (+): "))
b4 = int(input("enter value of b (+): "))
c4 = int(input("enter value of c (+): "))
lhs4 = (a4 + b4) + c4
rhs4 = a4 + (b4 + c4)
print("LHS=", lhs4)
print("RHS=", rhs4)
if lhs4 == rhs4:
    print("Associative law is True")
else:
    print("Associative law is false")
```

```
# Distributive law: multiplication over addition
a5 = int(input("Enter value a (dist mult/add): "))
b5 = int(input("enter value of b (dist mult/add): "))
c5 = int(input("enter value of c (dist mult/add): "))
lhs5 = a5 * (b5 + c5)
rhs5 = (a5 * b5) + (a5 * c5)
print(lhs5)
print(rhs5)
if lhs5 == rhs5:
    print("Distributive law is True")
else:
    print("Distributive law is false")
```

```
# Distributive law: Addition over multiplication
a6 = int(input("Enter value of a (dist add/mult): "))
b6 = int(input("enter value of b (dist add/mult): "))
c6 = int(input("enter value of c (dist add/mult): "))
lhs6 = a6 + (b6 * c6)
rhs6 = (a6 + b6) * (a6 + c6)
if lhs6 == rhs6:
    print("Distributive law is True")
else:
    print("Distributive law is false")
```

```

# Distributive law: AND over OR
a7 = bool(int(input("enter value a (dist AND/OR): ")))
b7 = bool(int(input("enter value b (dist AND/OR): ")))
c7 = bool(int(input("enter value c (dist AND/OR): ")))
lhs7 = a7 and (b7 or c7)
rhs7 = (a7 and b7) or (a7 and c7)
print(lhs7)
print(rhs7)
print("Checking Distributive law")
if lhs7 == rhs7:
    print("Distributive law is True")
else:
    print("Distributive law is false")

# Distributive law: OR over AND
a8 = bool(int(input("Enter value of a (dist OR/AND): ")))
b8 = bool(int(input("Enter value of b (dist OR/AND): ")))
c8 = bool(int(input("Enter value of c (dist OR/AND): ")))
lhs8 = a8 or (b8 and c8)
rhs8 = (a8 or b8) and (a8 or c8)
print("LHS=", lhs8)
print("RHS=", rhs8)
if lhs8 == rhs8:
    print("Distributive law is True")
else:
    print("Distributive law is false")

```